

Compressió Huffman amb Modes Adaptatius

Pràctica 4

Serafí Nebot Ginard
Universitat de les Illes Balears (UIB)
Grau en Enginyeria Informàtica
Algorismes Avançats, Curs 2025–2026

Resum—Aquesta memòria descriu el disseny i la implementació d'una aplicació Java per comprimir i descomprimir fitxers mitjançant codificació de Huffman. El programa analitza la distribució de freqüències del fitxer, construeix arbres de Huffman amb diferents estructures de cua de prioritat i tria automàticament entre modes d'1 byte, 2 bytes i compressió per blocs. La interfície gràfica permet seleccionar fitxers, visualitzar l'arbre generat, consultar la taula de codis i executar comparatives de rendiment. També s'ha implementat una CLI que reutilitza el mateix model que la GUI, de manera que els benchmarks són reproduïbles sense interacció manual. Es presenten proves unitàries centrades en round-trip, robustesa del format, correcció de cues i funcionament del benchmark, i un anàlisi experimental sobre els fitxers de Silesia.

Termes d'índex—Huffman, compressió sense pèrdues, entropia, cua de prioritat, MVC, Java Swing, benchmark, Silesia

1. Introducció

La codificació de Huffman és una tècnica de compressió sense pèrdues: el fitxer descomprimit ha de ser exactament igual que l'original. La idea principal és senzilla: els símbols que apareixen més sovint reben codis més curts, i els símbols menys freqüents reben codis més llargs. Aquests codis formen un codi prefix, és a dir, cap codi complet és prefix d'un altre; per això la descompressió pot llegir el flux de bits sense separadors explícits entre símbols [1].

L'objectiu de la pràctica no és només implementar Huffman, sinó construir una aplicació completa al seu voltant. El programa ha de poder comprimir fitxers de text o binaris, guardar un arxiu propi que es pugui descomprimir en una execució futura, mostrar l'arbre i la taula de codis, donar estadístiques de compressió i mesurar rendiment. A més, s'han comparat diferents estructures de cua de prioritat, perquè la construcció de l'arbre depèn directament d'extreure repetidament els dos nodes de menor freqüència.

Una decisió important d'aquesta implementació és que la compressió no produeix cap fitxer si l'arxiu resultant seria més gran que l'original. Això fa que alguns fitxers petits o molt aleatoris fallin de forma controlada: és preferible informar l'usuari que generar un resultat que va en contra del propòsit del propi programa.

2. Fonaments

2.1. Freqüències, Entropia i Codis

Abans de comprimir, el programa compta quantes vegades apareix cada símbol. Un símbol pot ser un byte individual

o una parella de bytes, segons el mode que s'estigui analitzant. Aquesta taula de freqüències és la base de tot el procés: defineix quins símbols mereixen codis curts i quins poden tenir codis més llargs.

L'entropia de Shannon dóna una estimació del nombre mínim de bits per símbol que es podria esperar per a una distribució concreta:

$$H = - \sum_i p_i \log_2(p_i)$$

on p_i és la probabilitat del símbol i . En aquesta pràctica s'utilitza com a valor de referència i com a estimació ràpida en la selecció de mida de bloc. No es modifica l'arbre directament amb l'entropia; l'arbre es construeix amb l'algorisme de Huffman clàssic. La diferència és que el programa calcula diversos plans possibles i selecciona el que promet una mida final més petita.

La longitud mitjana del codi es calcula com:

$$L = \frac{\text{bits totals codificats}}{\text{nombre de símbols}}$$

Per als modes de 2 bytes, el símbol és una parella de bytes. Per això els valors d'entropia i longitud mitjana s'han d'interpretar sempre dins del mateix tipus de símbol; no és equivalent comparar directament “bits per byte” i “bits per parella”.

2.2. Construcció de l'Arbre de Huffman

L'arbre es construeix de baix cap amunt. Primer es crea una fulla per a cada símbol amb freqüència major a 0. Després es repeteix el mateix pas fins que només queda

un node: extreure els dos nodes de menor freqüència, unir-los en un node intern i tornar-lo a inserir a la cua de prioritat. El node final és l'arrel de l'arbre [2].

Un detall pràctic important és el desempat. Quan dos símbols tenen la mateixa freqüència, hi pot haver diversos arbres òptims. La implementació ordena també pel símbol mínim de cada subarbre. Això no millora la mida de compressió, però fa que totes les estructures de cua generin el mateix arxiu, cosa essencial per provar que les cues són equivalents.

2.3. Diversos Modes de Compressió

El fitxer es pot mirar amb diferents “ulleres”. En mode d'1 byte, cada valor entre 0 i 255 és un símbol. En mode de 2 bytes, cada parella consecutiva és un símbol entre 0 i 65535. Si el fitxer conté patrons repetits de dos bytes, aquest segon mode pot ser clarament millor.

També hi ha un mode per blocs. Un sol arbre global pot ser dolent quan el fitxer canvia de comportament al llarg del temps. Per exemple, una part pot estar dominada per un byte i una altra per un altre. El mode per blocs divideix el fitxer i pren una decisió local per a cada bloc: Huffman d'1 byte, Huffman de 2 bytes o bloc emmagatzemat sense comprimir. El mode *STORED* només s'utilitza en aquest últim cas, com a fallback local. No s'ofereix com a compressió de fitxer sencer perquè només afegiria capçalera i faria créixer l'arxiu.

3. Descripció de l'Aplicació

L'aplicació té dues entrades: una interfície gràfica Swing i una línia de comandes. Les dues reutilitzen les mateixes classes del model, de manera que comprimir des de la GUI o des de la CLI executa el mateix procés intern.

3.1. Interfície Gràfica

La finestra principal està organitzada en quatre pestanyes.

- **Fitxers:** explorador integrat, ruta d'entrada, ruta de sortida, selector de cua de prioritat, selector de mode i resum d'estadístiques.
- **Taula de codis:** llista els símbols, valor hexadecimal, freqüència, probabilitat i codi assignat. Les columnes són ordenables, i les freqüències/probabilitats s'ordenen numèricament.
- **Arbre:** visualització de l'arbre de Huffman amb zoom i desplaçament. En mode per blocs es pot seleccionar quin bloc es vol inspeccionar.
- **Comparatius:** executa benchmarks sobre un directori de fitxers i mostra gràfiques de temps i taxa de compressió.

Durant la compressió o descompressió, la barra inferior mostra la fase actual, el percentatge, els bytes processats i una estimació del temps restant. Les operacions llargues s'executen amb *SwingWorker* per no bloquejar la interfície.

Imatge pendent

Captura suggerida: pestanya “Fitxers” amb un fitxer de Silesia seleccionat, estadístiques visibles i barra de progrés després d'una compressió.

Figura 1. Vista principal de l'aplicació.

3.2. CLI

La CLI exposa tres ordres:

- `compress <input> <output>`
- `decompress <input> <output>`
- `benchmark <inputDir> <repetitions> <outputCsv>`

La tercera ordre és la que s'ha usat per a les mesures de la Secció 6. La sortida CSV permet repetir proves, filtrar resultats i actualitzar taules sense dependre d'una sessió manual de GUI.

3.3. Ús Habitual

El flux normal d'ús és el següent. L'usuari selecciona un fitxer des de l'explorador integrat o escriu la ruta manualment. Si el fitxer no acaba en `.hff`, la vista prepara automàticament una ruta de sortida afegint aquesta extensió i el botó principal queda en mode compressió. Si el fitxer d'entrada ja és un arxiu `.hff`, la vista canvia el botó a mode extracció i proposa com a sortida el nom sense l'extensió.

Abans de comprimir, l'usuari pot triar la cua de prioritat i el mode de compressió; per defecte s'utilitza heap binari i mode automàtic. La resta d'opcions existeixen per experimentar i per poder comparar estructures, no perquè sempre siguin millors.

Quan acaba l'operació, la pestanya de fitxers mostra les estadístiques principals: mode efectiu, cua emprada, mida original, mida final, metadades, càrrega, entropia, longitud mitjana i temps. Si el resultat és per blocs, també mostra quants blocs han acabat en 1 byte, 2 bytes o emmagatzemats. Les altres pestanyes permeten entrar en el detall: taula de codis, arbre i blocs individuals.

4. Disseny i Implementació

4.1. Flux de Compressió i Descompressió

Els dos fluxos principals segueixen la mateixa idea: una seqüència de passes petites on cada fase prepara la informació que necessita la següent.

Compressió

- 1) Comptar freqüències i construir plans candidats.
- 2) Estimar la mida final de cada candidat, incloent capçalera, taula de freqüències i bits de càrrega.
- 3) Seleccionar el mode demanat o, en *AUTO*, el més petit.

- 4) Rebutjar la compressió si la mida estimada supera l'original.
- 5) Escriure l'arxiu.
- 6) Comprovar la mida real i eliminar la sortida si finalment és més gran que l'original.

Descompressió

- 1) Llegir la capçalera de l'arxiu `.hff`.
- 2) Validar el format: identificador, mode, mida original i freqüències.
- 3) Reconstruir la taula de freqüències i el mateix arbre de Huffman.
- 4) Llegir les dades comprimides bit a bit i caminar per l'arbre fins arribar a fulles.
- 5) Escriure els símbols originals fins arribar exactament a la mida original.
- 6) Rebutjar l'arxiu si falta informació, sobren dades obligatòries o les mides reconstruïdes no quadren.

La descompressió no reutilitza cap estat de la compressió anterior. Això és important per al requisit de poder comprimir un dia i descomprimir un altre: l'arxiu conté la capçalera, les freqüències i les metadades necessàries per reconstruir l'arbre. Com que la construcció és determinista, l'arbre reconstruït és equivalent al que es va usar durant la compressió. En mode `HUFFMAN_BLOCK`, el lector repeteix aquest procés per a cada bloc: primer llegeix la mida i el mode local del bloc, i després descodifica els bits d'aquell bloc amb l'arbre corresponent, o copia les dades si el bloc és `STORED`. La mida original guardada a la capçalera també indica quan s'ha d'aturar la lectura.

Els casos amb un únic símbol es tracten de forma especial. Si només hi ha un símbol possible, no fa falta escriure cap bit de càrrega: la freqüència ja diu quantes vegades s'ha de repetir aquell símbol.

Imatge pendent

Diagrama suggerit: entrada → anàlisi de freqüències → plans candidats → selector → writer; per a la descompressió, reader → reconstrucció d'arbre → sortida restaurada.

Figura 2. Flux intern de compressió i descompressió.

4.2. Modes de Compressió

La Taula 1 resumeix els modes disponibles. Més endavant s'expliquen més detalladament.

Taula 1. MODES DE COMPRESSIÓ IMPLEMENTATS.

Mode	Ús
AUTO	Analitza tots els modes Huffman i tria l'arxiu estimat més petit.
HUFFMAN_1_BYTE	Un arbre global sobre bytes individuals.
HUFFMAN_2_BYTE	Un arbre global sobre parelles de bytes; guarda el byte final separat si la mida és senar.
HUFFMAN_BLOCK	Divideix el fitxer en blocs i tria localment entre 1 byte, 2 bytes i emmagatzemat.
HUFFMAN_BLOCK_1_BYTE	Mode per blocs restringit a Huffman d'1 byte o emmagatzemat.
HUFFMAN_BLOCK_2_BYTE	Mode per blocs restringit a Huffman de 2 bytes o emmagatzemat.
STORED	Només intern dins un bloc o per llegir arxius antics; no es tria per a fitxer sencer.

El mode `AUTO` no prova a comprimir físicament el fitxer diverses vegades. Primer construeix plans d'anàlisi. Un pla conté la taula de freqüències, l'arbre, la taula de fulles per codificar ràpidament, el nombre estimat de bits i la mida de les metadades. Amb això es pot estimar la mida final abans d'escriure cap byte.

Per als modes globals, l'estimació és directa: mida de la capçalera més bits de Huffman. Per al mode per blocs, la mida estimada és la suma de la capçalera global i cada una de les millor opcions per a cada bloc.

Aquesta estratègia té dos avantatges. Primer, evita escriure arxius temporals només per comparar modes. Segon, permet aplicar el requisit de "no créixer" abans de tocar el fitxer de sortida. Tot i així, després d'escriure es fa una segona comprovació amb la mida real.

4.3. Mode per Blocs

El mode per blocs intenta resoldre el problema dels fitxers heterogenis: parts del fitxer molt diferents que poden aprofitar de tenir un arbre Huffman diferent. Primer s'estima una mida de bloc entre diverses candidates, totes múltiples de 4 KiB (4 KiB, 16 KiB, 64 KiB, 256 KiB, 1 MiB, 4 MiB i 16 MiB). Fer una compressió real per a cada candidata seria massa car; per això es fa una sola passada d'anàlisi i s'estima el cost de cada bloc amb la seva entropia i amb la mida de les metadades.

Per a cada bloc es comparen tres opcions:

- guardar-lo sense comprimir (`STORED`),
- comprimir-lo amb Huffman d'1 byte,
- comprimir-lo amb Huffman de 2 bytes.

Una opció Huffman només s'accepta si guanya el cost d'emmagatzemar el bloc directament. Això evita que la taula de freqüències faci créixer blocs petits o poc compressibles. El resultat final continua passant per la comprovació global de mida, de manera que els blocs `STORED` no poden provocar que es generi un arxiu comprimit més gran que l'original.

4.4. Format d'Arxiu

L'arxiu propi comença amb una capçalera. Aquesta capçalera és el motiu pel qual es pot tancar el programa després de comprimir i descomprimir en una altra execució: l'arxiu conté la informació necessària per reconstruir l'arbre.

Taula 2. ESTRUCTURA SIMPLIFICADA DE L'ARXIU .HUFF.

Part	Contingut
Magic	Bytes HUFF, per detectar que el fitxer és del format esperat.
Mode	Identificador del mode escrit.
Mida original	Nombre de bytes que s'han de restaurar. També evita llegir bits extra com si fossin símbols reals.
Freqüències globals	Només en modes de fitxer sencer. Es guarden com a taula: símbol i freqüència.
Blocs	En mode per blocs, cada bloc guarda mida, mode, freqüències si cal i dades comprimides.

4.5. Recorregut de Classes

Aquesta secció descriu les classes principals de la implementació. No s'enumeren només per paquet: s'explica també com cooperen durant una compressió, una descompressió i un benchmark.

4.5.1. Punts d'Entrada: Main i CLI. Main és el punt d'entrada gràfic. La seva feina és mínima: arrenca Swing al fil d'interfície, aplica l'aspecte visual del sistema, crea una MainWindow, connecta un Controller i mostra la finestra. No conté cap decisió de compressió. Aquesta simplicitat és intencionada: si la classe d'entrada comença a crear arbres o llegir fitxers, la separació MVC es perd des del principi.

CLI és el punt d'entrada per línia de comandes. Implementa un mètode `run()` que retorna un codi de sortida. Això permet provar-la sense executar un procés extern: les proves unitàries li passen els arguments i els streams en memòria com a paràmetres. Internament fa el mateix que la GUI: crea un `HuffmanCodec` i invoca compressió, descompressió o benchmark. La CLI no té una versió alternativa de l'algorisme, només un adaptador textual damunt el model.

Per mostrar progrés, la CLI defineix dos petits impressors interns. Un formatat `ProgressSnapshot` de compressió/descompressió i l'altre `BenchmarkProgressSnapshot`. Tots dos escriuen sobre `stderr` amb retorn de carro, de manera que la línia de progrés s'actualitza sense omplir la terminal. Quan l'operació acaba, la CLI imprimeix a `stdout` les mateixes dades que la GUI mostra al panell d'estadístiques: mode, cua, mides, entropia, longitud mitjana i temps.

4.5.2. Controller i ViewListener. `ViewListener` és la interfície que la vista utilitza per comunicar intencions: comprimir, descomprimir o executar comparatives. La vista no coneix la classe `Controller`; només guarda una referència a aquesta interfície.

`Controller` implementa `ViewListener`. En rebre una petició, fa validacions bàsiques de ruta, neteja els resultats anteriors, deshabilita controls i crea un `SwingWorker`. La part costosa viu a `doInBackground()`, on es crea el `HuffmanCodec` amb la cua i el mode triats. El codec rep un `ProgressListener`; cada vegada que el model emet una mostra de progrés, el worker la publica. Després, `process()` s'executa al fil de Swing i actualitza la barra de progrés. Finalment, `done()` recupera el resultat i crida `showCompressionInfo()`, `showDecompressionInfo()` o `showBenchmarkReport()`.

Aquesta estructura evita dos problemes habituals. Primer, el compressor no bloqueja la interfície. Segon, el model no actualitza components Swing directament; només emet dades de progrés. El controlador fa de frontera entre el treball de fons i la vista.

4.5.3. Vista: MainWindow i Panells. `MainWindow` és la finestra principal. Construeix les quatre pestanyes, manté els camps de ruta, els selectors de cua/mode, la taula de codis, el panell d'arbre, els panells de blocs i el panell de benchmark. Quan rep un `CompressionInfo`, no torna a calcular res: simplement transforma les dades en text, files de taula i nodes visuals.

La taula de codis usa un model propi, `SymbolTableModel`, amb tipus de columna concrets. Això és important perquè les columnes de freqüència i probabilitat s'ordenen numèricament, no com a text, i perquè els símbols es mostren de forma llegible sense perdre el valor real que necessita l'ordenació.

`FileBrowserPanel` implementa l'explorador integrat. Té un arbre de directoris i una taula de fitxers. La taula usa comparadors propis per ordenar nom, tipus, mida i data de modificació. En fer doble clic sobre un directori, navega; en fer doble clic sobre un fitxer, avisa a `MainWindow` per a que carregui la ruta seleccionada com a fitxer a processar. També usa un `WatchService` per refrescar el directori actual quan canvia el contingut, cosa útil després de crear un arxiu .huff.

`HuffmanTreePanel` dibuixa l'arbre. El model li passa un `HuffmanTreeNodeInfo`; el panell calcula una disposició jeràrquica recursiva i pinta arestes 0/1 i caixes de nodes. Manté escala i desplaçament propis per permetre zoom amb roda de ratolí i arrossegament. No coneix `HuffmanCode`; només coneix la representació d'informació preparada pel model.

`BlockListPanel` mostra una llista lateral de blocs en mode `HUFFMAN_BLOCK`. Cada cel·la indica número, mode local i mida, amb colors diferents per 1 byte, 2 bytes i emmagatzemat. Quan l'usuari tria un bloc, la finestra rep el `BlockInfo` corresponent i actualitza la taula de codis i l'arbre amb les dades d'aquell bloc.

`BenchmarkPanel` conté el formulari de directori/repeticions, la barra de progrés, el botó d'exportació CSV i les pestanyes de gràfiques. `BenchmarkGraphPanel` és un panell de dibuix 2D simple: rep sèries de punts, calcula

límits, genera marques d'eix amb `ChartAxisSupport` i pinta línies, punts, etiquetes i llegenda.

4.5.4. Format: `CompressionMode` i `ArchiveHeader`.

`CompressionMode` concentra tots els noms de mode en un únic enum. No són tipus Java separats, però no tots els valors són vàlids en tots els punts del programa. Per això la classe defineix mètodes de context, com `requestModes()`, `isArchiveMode()` i `allowedBlockHuffmanModes()`, que indiquen on es pot usar cada valor.

Els modes que es poden escriure a la capçalera .hff són `STORED`, `HUFFMAN_1_BYTE`, `HUFFMAN_2_BYTE` i `HUFFMAN_BLOCK`. Aquests són els únics amb identificador binari, perquè són els que el lector ha de trobar quan obre un arxiu. En canvi, `AUTO`, `HUFFMAN_BLOCK_1_BYTE` i `HUFFMAN_BLOCK_2_BYTE` són ordres de compressió: serveixen per dir al selector què ha d'analitzar, però no descriuen un format nou dins el fitxer.

Aquesta distinció és necessària encara que tot visqui dins la mateixa classe. `AUTO` pot aparèixer a la GUI perquè és una opció útil per a l'usuari, però no pot arribar al writer com a mode de capçalera perquè un lector futur no sabria quin arbre reconstruir. Els submodes `HUFFMAN_BLOCK_1_BYTE` i `HUFFMAN_BLOCK_2_BYTE` només restringeixen l'anàlisi dels blocs; si guanyen, l'arxiu continua guardant `HUFFMAN_BLOCK`. Finalment, `STORED` és serialitzable perquè pot aparèixer com a bloc local emmagatzemat o en arxius antics, però no s'ofereix com a compressió de fitxer sencer perquè sempre afegiria capçalera i faria créixer el resultat.

`ArchiveHeader` escriu i llegeix la capçalera binària. Guarda quatre bytes que identifiquen la capçalera (*magic*), el mode, la mida original i, si el mode és global, una taula de freqüències. No guarda sempre les 256 o 65.536 freqüències; escriu només les entrades presents (tenen freqüència major a 0). En llegir, valida la capçalera abans de deixar continuar: *magic* incorrecte, mode desconegut, freqüències duplicades o totals incoherents generen `ArchiveFormatException`.

4.5.5. `FrequencyTable` i `HuffmanCode`.

`FrequencyTable` és la representació bàsica de les freqüències. Internament usa un array `long[]` de mida fixa: 256 posicions per bytes o 65.536 per parelles. Aquesta decisió consumeix més memòria que un mapa en alguns casos, però dóna accés directe $O(1)$ i fa molt simple calcular mides, entropia i taules de capçalera. A més guarda `totalCount` i `distinctSymbolCount`; així no cal recórrer tot l'array cada vegada que es vol saber quants símbols diferents hi ha.

El mètode `addSymbol()` incrementa la freqüència del símbol. Si la freqüència anterior era zero, també incrementa `distinctSymbolCount`. Aquesta comprovació serveix per mantenir actualitzat el nombre de símbols diferents sense haver de recórrer tot l'array després de cada bloc o estimació. Aquest comptador s'utilitza per saber si hi ha el cas especial d'un únic símbol i per calcular ràpidament el cost de guardar la taula de freqüències a la capçalera.

`entropy()` calcula l'entropia de la distribució observada. `encodedBitCount()` multiplica la freqüència de cada símbol per la profunditat de la seva fulla de Huffman i obté el nombre exacte de bits de càrrega. `clear()` permet reutilitzar la mateixa taula en la selecció de mida de bloc, evitant crear repetidament arrays de 65.536 posicions.

`HuffmanCode` representa tant fulles com nodes interns de l'arbre. Una fulla conté un símbol real i una freqüència. Un node intern conté els dos fills i la suma de freqüències. Quan dos nodes tenen la mateixa freqüència, `compareTo()` desempata pel símbol mínim del subarbre. Amb això, l'arbre és determinista independentment de l'implementació de la cua de prioritat.

La construcció de l'arbre es fa a `buildFromFrequencies()`. El mètode rep una `FrequencyTable` i una `NodeQueue` que és l'interfície comuna de les cues de prioritat del projecte: l'algorisme només necessita inserir nodes, extreure el mínim i consultar la mida, independentment de si al darrere hi ha un heap binari, una llista ordenada o un heap de Fibonacci.

Primer, per cada símbol amb freqüència positiva es crea una fulla amb `single()` i s'insereix a la cua. Aquest mètode rep el símbol i la seva freqüència, però només construeix un node fulla: símbol real, freqüència i cap fill. Després l'algorisme extreu els dos nodes amb menor freqüència i els passa a `merge()`, que crea un node intern nou que té aquests dos nodes com a fills i freqüència igual a la suma de les dues freqüències. Aquest node representa el subarbre combinat i es torna a inserir a la cua. Repetint aquest procés, la cua va passant de moltes fulles petites a subarbres cada vegada més grans fins que només queda una arrel.

El codi binari de cada fulla no es construeix durant aquest procés. Es calcula després de forma mandrosa amb una única passada en amplada: cada fill afegeix un 0 o un 1 al codi del pare i guarda també la profunditat. Finalment, `lookupBySymbol()` crea una taula símbol $\rightarrow$ fulla perquè el writer pugui codificar bytes sense recórrer l'arbre per cada símbol.

4.5.6. Cues de Prioritat. L'algorisme de Huffman només necessita tres operacions de les cues: inserir, extreure el mínim i consultar la mida. Per això el model defineix una interfície mínima `NodeQueue`. El codi que construeix l'arbre no sap quina estructura concreta hi ha darrere d'aquesta interfície; només usa aquestes tres operacions.

`PriorityQueueStrategy` és l'enum que connecta la selecció de l'usuari amb la implementació real. La GUI i el benchmark treballen amb aquests valors, com `BINARY_HEAP`, `DICHOTOMIC_LIST` i `FIBONACCI_HEAP`. Quan el codec necessita construir un arbre, la estratègia crea la `NodeQueue` corresponent amb `createQueue()`. Així el selector visible a la interfície, el benchmark i el codi de Huffman comparteixen una sola llista de cues disponibles.

Les tres implementacions són:

- **BinaryHeapNodeQueue:** estratègia per defecte, basada en una `PriorityQueue` de Java. Interna-

ment aquesta estructura manté els elements en un heap binari: el mínim queda sempre a l'arrel i, quan s'insereix o s'elimina, es reordena només el camí necessari per conservar la propietat de heap. Té bon rendiment pràctic, poca lògica pròpia i cost $O(\log n)$ per inserció i extracció.

- **DichotomicListNodeQueue:** manté una llista ordenada amb `ArrayList`. Quan s'afegeix un node, usa cerca binària per trobar la posició que li correspon segons l'ordre de Huffman i l'insereix en aquell punt. Això fa que extreure el mínim sigui directe, perquè el mínim és al principi de la llista. El cost amagat és el desplaçament dels elements de l'array: trobar la posició és ràpid, però inserir pot costar $O(n)$.
- **FibonacciHeapNodeQueue:** estructura més complexa, amb insercions molt barates i extracció mínima amortitzada logarítmica [5]. La implementació manté una llista circular d'arrels i cada node pot tenir una llista circular de fills. Inserir és gairebé només afegir una arrel nova i actualitzar el punter al mínim. En extreure el mínim, els seus fills passen a la llista d'arrels i després es fa una consolidació: si dues arrels tenen el mateix grau, s'uneixen i la de menor prioritat queda com a pare.

4.5.7. Plans Interns i HuffmanArchive. `CodecPlans` no conté lògica; conté tres *records* interns que fan de frontera entre anàlisi i escriptura.

`WholePlan` descriu un mode global complet: freqüències, arrel, fulles, bits estimats, entropia, longitud mitjana, capçalera i mida estimada. S'usa tant per al mode d'1 byte com per al de 2 bytes.

`BlockUnit` descriu un bloc concret dins el mode per blocs. Guarda la mida del bloc, el mode local seleccionat i les dades que el writer necessitarà. Pot contenir la taula/arbre d'1 byte, la taula/arbre de 2 bytes o cap arbre si el bloc ha quedat en mode `STORED` emmagatzemat. La raó és que l'analitzador ja ha hagut de construir aquests arbres per saber quants bits ocuparia cada alternativa i decidir el mode local del bloc. Si el writer només guardàs el mode i les freqüències, hauria de tornar a construir el mateix arbre durant l'escriptura. Això duplicaria temps de càlcul i faria que l'escriptura depengués d'una segona reconstrucció que no aporta cap decisió nova. Amb `BlockUnit`, el writer rep el pla tancat: només serialitza exactament el que l'anàlisi ha triat.

`BlockPlan` agrupa tots els `BlockUnit` i afegeix dades agregades del mode per blocs: mida de bloc triada, capçalera, metadades, mida estimada, bits totals Huffman, entropia mitjana ponderada i longitud mitjana.

`HuffmanArchive` representa l'arxiu Huffman en el punt on ja se sap quin format s'ha triat. Durant la compressió, els analitzadors poden generar diversos plans candidats, però només un s'acabarà escrivint. `HuffmanArchive` empaqueta aquesta decisió: guarda el mode final, la capçalera i el pla concret que correspon a aquell mode. Si el mode és global, conté un `WholePlan`; si és per blocs, conté un

`BlockPlan`. Això evita passar per separat un mode, una capçalera i un pla opcional a cada mètode del writer.

La mateixa classe també s'utilitza després de llegir un arxiu. En aquest cas no conté un pla de compressió, perquè la decisió ja havia estat presa quan es va crear el fitxer. Conté la capçalera llegida i la informació reconstruïda que necessita la GUI: símbols, arbre i blocs. Per això `HuffmanArchive` és una frontera comuna entre el codec i el format: abans d'escriure diu "això és el que s'ha de serialitzar", i després de llegir diu "això és el que s'ha reconstruït del fitxer".

4.5.8. Analitzadors i Selector. `HuffmanCodec` és una façana. Manté l'API pública que usen la GUI, la CLI i el benchmark, però delega les parts concretes: `HuffmanWholeFileAnalyzer` analitza modes globals, `HuffmanBlockAnalyzer` analitza blocs, `HuffmanModeSelector` tria el pla final, i `HuffmanArchiveWriter/Reader` només escriuen o llegeixen el format.

La raó d'aquesta divisió és evitar que la classe principal del codec barregi tres preguntes diferents: "quant ocuparia cada opció?", "quina opció convé escriure?" i "com s'escriu exactament aquesta opció?". Si aquestes decisions viuen juntes, el codi sembla més curt al principi però és més difícil d'explicar i provar. Separant-les, cada classe té una responsabilitat que coincideix amb una part del problema.

`HuffmanAnalyzer` és una superclasse petita per als analitzadors. Només centralitza `buildTree()`: construeix l'arbre amb la cua triada i acumula el temps invertit en aquesta construcció per tal de poder comparar el seu rendiment. No força cap flux d'anàlisi comú, perquè analitzar un fitxer sencer i analitzar blocs són processos diferents.

`HuffmanWholeFileAnalyzer` construeix els plans globals. `analyzeByte()` llegeix el fitxer en buffers i afegeix cada byte a una `FrequencyTable` de 256 símbols. `analyzeWord()` fa el mateix amb parelles de bytes. En aquest cas, "buffer" vol dir buffer d'entrada de Java, no bloc de compressió: és només la manera de llegir el fitxer a trossos. Si una lectura acaba just després del primer byte d'una parella, aquest byte no es pot comptar encara com a símbol de 16 bits; es guarda a `pendingByte`. En llegir el buffer següent, el primer byte nou es combina amb aquest pendent i la parella ja es pot afegir a la taula de freqüències. Si el fitxer acaba i encara queda un `pendingByte`, vol dir que la mida original és senar: aquest byte final no forma cap parella i es guarda separat al pla perquè el writer l'escrigui cru al final del flux Huffman. Després de comptar, construeix l'arbre, calcula fulles, bits, entropia, capçalera i mida estimada.

`HuffmanBlockSizeSelector` tria una mida de bloc abans de fer l'anàlisi complet. Les mides candidates són 4 KiB, 16 KiB, 64 KiB, 256 KiB, 1 MiB, 4 MiB i 16 MiB; totes són múltiples de 4 KiB. Per això el selector pot llegir el fitxer en mostres de 4 KiB i alimentar totes les candidates al mateix temps: la candidata de 4 KiB tanca un bloc a cada mostra, la de 16 KiB cada quatre mostres, la de 64 KiB cada setze, i així successivament. Cada candidata acumula freqüències fins omplir el seu bloc sintètic; llavors estima si convindria emmagatzemar, usar Huffman d'1 byte

o usar Huffman de 2 bytes. L'estimació usa entropia i cost de metadades, no arbres reals, perquè construir arbres per a totes les candidates seria massa car.

`HuffmanBlockAnalyzer` fa l'anàlisi complet del mode per blocs amb la mida ja seleccionada. Recorre el fitxer bloc a bloc i, per a cadascun, prova només els candidats Huffman que permet el mode demanat: `HUFFMAN_BLOCK` i `AUTO` permeten 1 byte i 2 bytes, `HUFFMAN_BLOCK_1_BYTE` només 1 byte, i `HUFFMAN_BLOCK_2_BYTE` només 2 bytes. `STORED` sempre queda disponible com a alternativa local.

Per cada candidat permès, l'analitzador calcula el cost del bloc: capçalera local, taula de freqüències i bits Huffman. En el candidat de 2 bytes també suma el byte final cru si el bloc té mida senar. Després compara aquests costos amb el cost de copiar el bloc directament. Si cap candidat Huffman és més petit que copiar-lo, el bloc es marca com a `STORED`. Cada bloc és independent: no es comparteixen parelles de 2 bytes ni arbres entre blocs.

`HuffmanModeSelector` és el punt on es converteix l'anàlisi en una decisió final. Abans d'arribar aquí, el codec ja ha calculat quant ocuparia cada alternativa possible: Huffman global d'1 byte, Huffman global de 2 bytes i Huffman per blocs. Si l'usuari ha demanat un mode concret, el selector simplement tria aquella alternativa i falla si no és aplicable. Per exemple, `STORED` no s'accepta com a mode de fitxer sencer perquè això generaria un fitxer "comprimat" més gran que l'original (dades originals + capçalera) cosa que no tendria sentit.

4.5.9. Writer i Reader. `HuffmanArchiveWriter` rep un `HuffmanArchive` ja triat i escriu exactament el pla seleccionat.

`BitOutputStream` és el suport per escriure bits. Acumula bits en un byte temporal i el descarrega quan està ple. Si l'últim byte queda incomplet, l'omple amb zeros. Aquests zeros són segurs perquè el decoder sap quants bytes originals ha de restaurar i s'atura quan arriba a aquesta mida.

`HuffmanArchiveReader` fa el camí invers. Llegeix `ArchiveHeader`, reconstrueix la taula de freqüències i torna a construir l'arbre. En modes Huffman, usa `BitInputStream` per llegir un bit cada vegada i caminar per l'arbre fins trobar una fulla. En mode per blocs, repeteix aquest procés per cada bloc i valida que la suma de bytes restaurats arribi exactament a la mida original.

4.5.10. Informació, Progrés i Utilitats. El paquet `info` conté dades immutables. `CompressionStats` i `DecompressionStats` són el resum numèric. `CompressionInfo` i `DecompressionInfo` combinen estadístiques amb taula de símbols, arbre i llista de blocs. `HuffmanSymbolInfo`, `HuffmanTreeNodeInfo` i `BlockInfo` són còpies preparades per mostrar: contenen només els camps que necessita la GUI, com símbol, freqüència, probabilitat, codi, fills de l'arbre, mida de bloc o mode local. La vista no rep les estructures internes que usa el codec per comprimir, sinó aquests objectes de només

lectura. `HuffmanInfoFactory` fa aquesta conversió, calculant codis com a cadenes i probabilitats per a la taula.

El paquet `progress` desacobla el model de la interfície. `ProgressPhase` enumera les fases visibles del treball, com analitzar, escriure o llegir. `ProgressTracker` rep comptadors de bytes i emet `ProgressSnapshot` cada 64 KiB o quan una fase acaba. L'estimació del temps restant és una extrapolació simple: temps transcorregut, bytes processats i bytes pendents. Si encara no hi ha prou informació, retorna -1 i la vista mostra "-".

`ByteFormat` només converteix mides de bytes a unitats llegibles per la GUI i la CLI.

4.5.11. Benchmark. `BenchmarkConfig` guarda la configuració d'una execució: directori d'entrada, nombre de repeticions i opcions seleccionades. `BenchmarkService` carrega tots els fitxers no comprimits en format `.hff` del directori, els ordena per mida i executa dues sèries. Durant l'execució, `BenchmarkProgressListener` rep mostres de progrés perquè la GUI o la CLI puguin mostrar l'estat sense que el servei conegui cap component visual.

La primera sèrie compara cues. Per cada fitxer i cada `PriorityQueueStrategy`, comprimeix en mode automàtic, descomprimeix el resultat i verifica amb `Files.mismatch()` que la sortida sigui idèntica. Després agrega els temps de construcció d'arbre de compressió i descompressió en `QueueBenchmarkPoint`. Aquesta comparativa només necessita aquests temps, perquè la taxa de compressió depèn del mode seleccionat i es mostra a la segona sèrie.

La segona sèrie compara modes. Usa heap binari i demana explícitament `HUFFMAN_1_BYTE`, `HUFFMAN_2_BYTE`, `HUFFMAN_BLOCK_1_BYTE` i `HUFFMAN_BLOCK_2_BYTE`. Mesura temps total de compressió i descompressió, i guarda les dades en `CompressionModeBenchmarkPoint`. Finalment, `BenchmarkReport` agrupa totes les files i `BenchmarkCsvWriter` les serialitza en dues seccions CSV.

4.5.12. Arquitectura MVC. La separació general segueix el patró Model-Vista-Controlador. Després de veure les classes principals, és més fàcil situar cada paquet dins d'aquesta arquitectura. En aquest projecte no és pràctic dibuixar un diagrama amb totes les classes perquè el model és bastant ampli; és més clar representar paquets i responsabilitats.

- **Model:** compressió, descompressió, format d'arxiu, cues de prioritat, progrés, informació per a la GUI i benchmark.
- **Vista:** components Swing, taules, arbre, explorador de fitxers i gràfiques.
- **Controlador:** valida rutes, crea el codec amb les opcions triades, executa treballs en segon pla i passa els resultats a la vista.

La vista no construeix arbres ni interpreta arxius. El controlador no sap com es dibuixa l'arbre. El model no importa classes de la GUI. Aquesta separació fa que la CLI pugui usar el mateix compressor sense crear cap finestra.

Imatge pendent

Diagrama suggerit per paquets, no per classes individuals: `view` i `CLI` apunten al `controller/HuffmanCodec`; el `codec` coordina `codec`, `archive`, `queue`, `progress` i `info`; `benchmark` reutilitza `HuffmanCodec`.

Figura 3. Arquitectura MVC proposada per a la documentació visual.

Taula 3. COBERTURA FUNCIONAL DE LES PROVES UNITÀRIES.

Grup	Comprovacions principals	Tests
HuffmanCodecTest	Round-trip amb bytes repetits, tots els valors de byte, text, mode de 2 bytes, mode per blocs, restriccions de submode, selecció automàtica, rebuig de fitxers buits o aleatoris que creixerien, determinisme amb freqüències iguals, progrés, errors de format, arxius truncats, totals de freqüència incorrectes, benchmark i CSV.	22
NodeQueueTest	Cada cua elimina en ordre creixent, es compara amb <code>PriorityQueue</code> sota 2.000 operacions mixtes aleatòries i falla correctament quan es demana eliminar el mínim d'una cua buida. Les mateixes proves s'apliquen al heap binari, la llista dicotòmica i el heap de Fibonacci.	9
HuffmanCodeTest	La taula de fulles permet accés directe a qualsevol símbol, inclòs 255, i els codis/profunditats es calculen de forma coherent i mandrosa.	2
CLITest	La CLI comprimeix i descomprimeix un fitxer correctament, i mostra ajuda quan els arguments són incorrectes.	2

5. Validació amb Proves Unitàries

Les proves s'executen amb JUnit 5 mitjançant `mvn test`. En el moment d'escriure aquesta memòria hi ha 35 proves i totes passen. La Taula 3 resumeix què cobreixen.

Les proves de round-trip són les més importants: escriuen dades originals, les comprimeixen, les descomprimeixen i comparen byte a byte el resultat. Això valida conjuntament el comptatge de freqüències, la construcció de l'arbre, l'escriptura de bits, la lectura de bits i el format d'arxiu.

Les proves de cues són independents del compressor. Això és intencionat: si una cua falla, l'arbre de Huffman podria quedar mal construït encara que la resta del compressor fos correcta. Comparar cada implementació amb `PriorityQueue` sota operacions barrejades dona una garantia forta que totes respecten el mateix contracte.

També hi ha proves negatives. Per exemple, es creen arxius amb capçalera incorrecta, mode no suportat o càrrega truncada, i s'exigeix que la descompressió falli. Això evita que el programa produeixi sortides silenciosament corruptes quan rep un fitxer invàlid.

6. Resultats i Discussió

6.1. Protocol de Benchmark

Per a aquesta primera versió s'ha utilitzat la carpeta `P4/res`, que conté els fitxers de Silesia. Silesia és un conjunt de 12 fitxers pensat per provar compressors sense pèrdues amb dades de naturalesa diferent: text, HTML, executables, bases de dades i imatges mèdiques. La mida total original és d'aproximadament 212 MB [3].

La comanda executada ha estat:

```
java -cp target/P4-1.0-SNAPSHOT.jar \
```

```
com.serafinebot.p4.CLI benchmark \
res 1 memoria/silesia-benchmark.csv
```

Listing 1. Benchmark CLI sobre Silesia.

La GUI usa exactament el mateix `BenchmarkService`. La diferència és que la CLI exporta directament el CSV i evita interferència manual. En aquesta execució s'ha usat una repetició per fitxer; per tant, els valors serveixen com a fotografia inicial, no com a estudi estadístic final.

El benchmark fa dues comparatives:

- **Cues de prioritat:** executa AUTO amb cada estructura i mesura el temps de construcció d'arbre en compressió i descompressió.
- **Modes de compressió:** amb heap binari, compara els modes Huffman d'1 byte, 2 bytes i blocs restringits.

Cada compressió es descomprimeix després i es compara amb l'original. Per tant, una fila de benchmark només és vàlida si també ha passat la verificació de descompressió.

6.2. Limitacions de la Mesura Actual

Les dades d'aquesta memòria són útils per comparar tendències, però no s'han d'interpretar com a valors absoluts definitius. Només s'ha executat una repetició per fitxer, de manera que hi pot haver soroll de la JVM, del sistema operatiu o de la cache de disc. En la versió final del treball seria convenient repetir cada cas diverses vegades i presentar mitjanes o medians.

També convé separar dues preguntes diferents. Una és quin mode comprimeix més. L'altra és quin mode és més ràpid. Un mode pot reduir molt la mida però ser més lent de descomprimir perquè l'arbre és més gran o perquè el

Taula 4. MILLOR MODE PER FITXER EN EL BENCHMARK SILESIA.

Fitxer	Mida (B)	Millor mode	Reducció (%)	Comp. / Descomp. (ms)
xml	5.345.280	HUFFMAN_BLOCK_2_BYTE	47,051	159,6 / 95,0
ooffice	6.152.192	HUFFMAN_BLOCK_1_BYTE	18,712	206,8 / 146,5
reymont	6.627.202	HUFFMAN_BLOCK_2_BYTE	49,824	191,8 / 107,7
sao	7.251.944	HUFFMAN_2_BYTE	12,316	415,4 / 314,7
x-ray	8.474.240	HUFFMAN_BLOCK_2_BYTE	29,840	337,7 / 255,9
mr	9.970.564	HUFFMAN_BLOCK_2_BYTE	61,019	304,8 / 142,3
osdb	10.085.684	HUFFMAN_2_BYTE	21,563	533,1 / 386,0
dickens	10.192.446	HUFFMAN_2_BYTE	49,375	323,4 / 212,6
samba	21.606.400	HUFFMAN_BLOCK_2_BYTE	35,748	876,6 / 422,5
nci	33.553.445	HUFFMAN_2_BYTE	72,365	891,5 / 266,8
webster	41.458.703	HUFFMAN_2_BYTE	47,301	1325,4 / 842,7
mozilla	51.220.480	HUFFMAN_2_BYTE	35,224	2404,8 / 1322,3

símbol és de 2 bytes. Per això les taules mostren percentatge i temps, i no només una única puntuació agregada.

Finalment, Silesia és més representatiu que fitxers artificials, però no cobreix tots els casos d'ús. Fitxers ja comprimits, dades criptogràficament aleatòries o fitxers molt petits probablement serien rebutjats pel compressor perquè la capçalera i les taules ocuparien més que el guany possible.

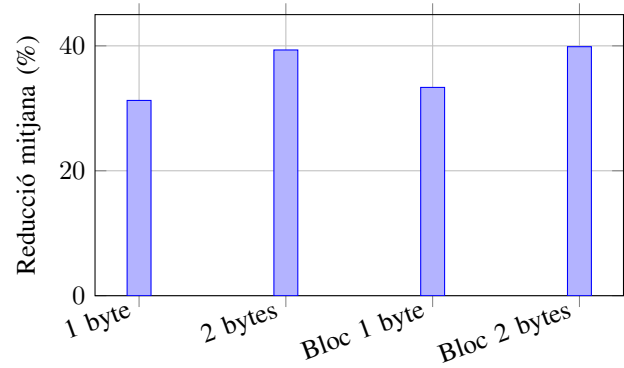


Figura 4. Reducció mitjana per mode de compressió sobre Silesia.

6.3. Resultats per Mode

La Taula 4 mostra el millor mode observat per a cada fitxer de Silesia. El percentatge indica reducció respecte a l'original: per exemple, 47% significa que l'arxiu final ocupa un 47% menys.

El mode de 2 bytes és molt competitiu en aquests fitxers. Això indica que molts fitxers tenen repeticions útils a nivell de parelles de bytes. El mode per blocs guanya especialment quan la distribució local varia: xml, reymont, x-ray, mr i samba milloren amb blocs de 2 bytes, mentre que ooffice prefereix blocs d'1 byte.

La mitjana confirma la intuïció: el mode de 2 bytes comprimeix millor, però també descomprimeix més lentament perquè l'espai de símbols és més gran i la lectura reconstrueix parelles. Els blocs d'1 byte són els més ràpids en compressió, perquè eviten arbres globals grans i poden saltar a emmagatzemat en blocs on Huffman no compensa.

6.4. Resultats per Cua de Prioritat

La Taula 6 compara només el temps de construir arbres. La taxa de compressió és idèntica perquè el desempat determinista fa que les tres cues produeixin els mateixos arbres finals.

Taula 5. MITJANES PER MODE SOBRE ELS 12 FITXERS.

Mode	Red. %	Comp. ms	Descomp. ms
HUFFMAN_1_BYTE	31,261	660,0	295,0
HUFFMAN_2_BYTE	39,340	694,7	393,0
BLOCK_1_BYTE	33,346	447,7	293,8
BLOCK_2_BYTE	39,866	663,5	383,4

Taula 6. TEMPS MITJÀ DE CONSTRUCCIÓ D'ARBRE PER CUA.

Cua	Comp. ms	Descomp. ms	Red. %
Heap binari	25,083	5,333	40,315
Heap de Fibonacci	128,583	26,917	40,315
Llista dicotòmica	419,000	92,500	40,315

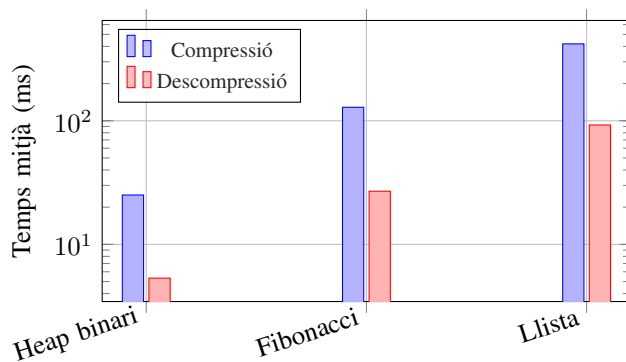


Figura 5. Temps mitjà de construcció de l'arbre segons la cua de prioritat.

El heap binari és clarament la millor opció pràctica per defecte. El heap de Fibonacci té bones propietats teòriques, però en aquesta aplicació la complexitat de l'estructura i la manipulació d'enllaços pesen més que el guany amortitzat. La llista dicotòmica és útil com a implementació simple i com a comparació, però els desplaçaments dins la llista la penalitzen.

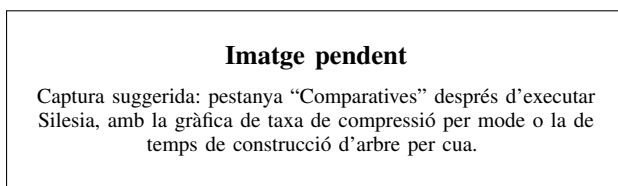


Figura 6. Gràfiques principals de benchmark.

Referències

- [1] GeeksforGeeks, "Huffman Coding | Greedy Algo-3," 2025. [Online]. Available: <https://www.geeksforgeeks.org/dsa/huffman-coding-greedy-algo-3/>
- [2] Wikipedia, "Huffman coding," 2026. [Online]. Available: https://en.wikipedia.org/wiki/Huffman_coding
- [3] S. Deorowicz, "Silesia compression data set," Silesian University of Technology. [Online]. Available: <https://sun.aei.polsl.pl/~sdeor/index.php?page=silesia>
- [4] Oracle, "Worker Threads and SwingWorker," The Java Tutorials. [Online]. Available: <https://docs.oracle.com/javase/tutorial/uiswing/concurrency/worker.html>
- [5] Wikipedia, "Fibonacci heap," 2026. [Online]. Available: https://en.wikipedia.org/wiki/Fibonacci_heap

7. Conclusions

La pràctica ha evolucionat cap a un compressor Huffman complet amb GUI, CLI, format propi i benchmark. La part més important del disseny és que el codec no implementa tots els detalls en una sola classe: analitzar, seleccionar, escriure, llegir i construir informació per a la GUI són responsabilitats separades. Això fa que el programa sigui més fàcil d'explicar, provar i modificar.

Els resultats inicials amb Silesia mostren que adaptar el símbol pot ser més rellevant que optimitzar només l'estructura de cua. El mode de 2 bytes i el mode per blocs de 2 bytes són els que millor comprimeixen en la majoria dels fitxers. En canvi, per a la cua de prioritat, el heap binari és el millor compromís pràctic malgrat que el heap de Fibonacci sigui més sofisticat teòricament.

Les proves unitàries cobreixen el camí crític: round-trip, format invàlid, determinisme, cues, CLI i benchmark. Això és especialment important en un compressor, perquè un error pot no aparèixer com una excepció immediata, sinó com un fitxer restaurat lleugerament diferent. Per això la comparació byte a byte és la validació central.

Com a treball futur, convé ampliar el benchmark amb més repeticions i amb altres conjunts de dades, regenerar les taules amb mitjanes més estables i afegir captures definitives de la GUI i dels diagrames proposats en aquesta memòria.